
Plataforma de Orquestación del Ciclo de Vida del Software

SLCP

Especificación normativa de trazabilidad, restricciones tecnológicas,
vocabulario canónico e interoperabilidad de artefactos

Documento **SLCP-DOC-001** – Versión **1.15**

6 de agosto de 2026

Este documento consolida las decisiones normativas adoptadas para la plataforma SLCP: el modelo de trazabilidad de extremo a extremo, las restricciones de construcción con tecnologías libres, las plataformas destino de generación de código, el vocabulario canónico en inglés con sus convenciones de nomenclatura, y los formatos de exportación e importación de artefactos. Cada afirmación no trivial está respaldada por una norma internacional o por una fuente pública verificable, listadas en la sección de referencias.

Cambios de la versión 1.1: incorpora la decisión registrada en SLCP-ADR-0002, que establece la generación del código fuente de la lógica de negocio como requisito del producto. Se amplía el marco normativo, se amplía el vocabulario canónico, se reescribe TRC-23, se añaden los requisitos TGT-08 a TGT-13 y se amplían los criterios de piso.

Cambios de la versión 1.2: incorpora el marco de niveles de SLCP-DOC-000. Cada requisito declara ahora el nivel al que aplica; se corrigen las conflaciones detectadas en NAM-01 y CON-01, y se añaden NAM-07, CON-06 y TGT-14.

Cambios de la versión 1.3: incorpora las entidades de organización del equipo y las familias de requisitos definidas en SLCP-DOC-004.

Cambios de la versión 1.4: incorpora la familia ADM y precisa TRC-24 con la distinción entre baja, eliminación y purga establecida en SLCP-ADR-0003.

Cambios de la versión 1.5: incorpora las entidades de planificación y recursos y las familias WBS y RES definidas en SLCP-DOC-005.

Cambios de la versión 1.6: añade la relación `realizes` al conjunto cerrado de TRC-02 y las familias PRG y MAP definidas en SLCP-DOC-006.

Cambios de la versión 1.7: adopta la denominación `ProjectMembership`, añade la relación `contributesTo` y las familias RQM, PLN, ANA y RPT de SLCP-DOC-007.

Cambios de la versión 1.8: resuelve la colisión del término componente, incorpora las entidades de especificación ejecutable y modelado, y añade las familias SPC, MOD y ARQ de SLCP-DOC-008.

Cambios de la versión 1.9: añade la familia PRO de propagación de cambios y la entidad de punto de vista arquitectónico, definidas en SLCP-DOC-009.

Cambios de la versión 1.10: subordina TGT-12 a la precedencia de verdad de PRO-15 y añade la entidad de anulación declarada, definidas en SLCP-DOC-010.

Cambios de la versión 1.11: añade la entidad de bloque protegido y extiende TRC-11 a su cómputo de huérfanos, conforme a SLCP-ADR-0004.

Cambios de la versión 1.12: añade la familia VER de estrategia de prueba y las referencias normativas de prueba de mutación y metamórfica, definidas en SLCP-DOC-011.

Cambios de la versión 1.13: añade NAM-08 a NAM-10 sobre validación de nombres propuestos por la persona, conforme a SLCP-DOC-013.

Cambios de la versión 1.14: añade las entidades de ejecución, resultado e instrumento de verificación definidas en SLCP-DOC-016.

Cambios de la versión 1.15: se fija la licencia MIT para la plataforma y se confirma Java con Spring Boot y PostgreSQL como primer destino de generación.

Índice

1	Propósito, alcance y convenciones	3
1.1	Propósito	3
1.2	Alcance	3
1.3	Convención de redacción de los requisitos	3
1.4	Niveles de abstracción	3
1.5	Sistema de identificación de los requisitos de este documento	4
2	Normalización terminológica aplicada	4
3	Marco normativo de referencia	5
4	Modelo de trazabilidad	5
4.1	Ejes de trazabilidad	6
4.2	Metamodelo	6
4.3	Identificación y versionado	6
4.4	Calidad de los requisitos como precondition	6
4.5	Cadena de enlaces	7
4.6	Cobertura, cambio e impacto	8
4.7	Procedencia de los artefactos generados	9
4.8	Persistencia y auditoría	9
5	Restricciones tecnológicas de construcción	10
5.1	Definición operativa de <i>libre</i>	10
5.2	Delimitación entre plataforma y destino	10
5.3	Higiene de licencias	11
5.4	Pila tecnológica de construcción	11
6	Plataformas destino de la generación	12
6.1	Criterio de selección	12
6.2	Lenguajes	12
6.3	Sistemas gestores de bases de datos	13
6.4	Frameworks	13
6.5	Generación de la lógica de negocio	14
7	Vocabulario canónico y convenciones de nomenclatura	15

7.1	Regla de idioma	15
7.2	Vocabulario canónico	15
7.3	Convenciones por artefacto	16
7.4	Colisión con palabras reservadas	17
7.5	Esquema de identificadores legibles	18
8	Interoperabilidad: exportación e importación	18
8.1	Principios	19
8.2	Formatos por tipo de artefacto	19
8.3	Requisitos de importación	19
8.4	Ida y vuelta y formato nativo	20
9	Criterios de aceptación consolidados	21
10	Asuntos abiertos	21
A	Palabras reservadas: fuentes de la lista de exclusión	22

1 Propósito, alcance y convenciones

1.1 Propósito

Este documento establece los requisitos normativos que la plataforma SLCP debe satisfacer en cuatro dimensiones interdependientes: trazabilidad, restricciones tecnológicas de construcción, plataformas destino de generación y nomenclatura e interoperabilidad de artefactos. Su función es servir de referencia única del proyecto, de modo que toda decisión posterior se contraste contra un requisito identificado en lugar de contra una conversación previa.

1.2 Alcance

SLCP es una plataforma que, a partir de una Especificación de Requisitos del Software (ERS) conforme a Int [1], orquesta y automatiza los procesos técnicos del ciclo de vida definidos en Int [2]: análisis, arquitectura, diseño, construcción, integración, verificación, validación y soporte al mantenimiento. Queda explícitamente fuera de alcance la sustitución del juicio humano en las transiciones de fase: cada paso de línea base exige aprobación humana registrada.

Nota sobre la edición vigente de 12207. La edición de referencia es Int [2], revisada y confirmada en 2023. ISO registra una segunda edición fechada en abril de 2026 en fase final de publicación, que reemplazará a la anterior [3]. Antes de declarar conformidad formal debe verificarse cuál edición está publicada en la fecha de la declaración.

1.3 Convención de redacción de los requisitos

Se aplica la convención verbal de Int [1]: **debe** expresa una obligación cuyo incumplimiento es una no conformidad; **debería** expresa una recomendación cuya omisión requiere justificación registrada; **puede** expresa una opción sin implicación normativa. Todo requisito de este documento incluye un criterio de verificación objetivo, porque un requisito sin criterio de verificación no es verificable y, por definición de la propia norma, no es un requisito bien formado.

1.4 Niveles de abstracción

SLCP es una aplicación que genera aplicaciones, y por tanto cada requisito puede referirse a tres objetos distintos. SLCP-DOC-000 define esos niveles: [N1] es la plataforma SLCP; [N2] es el proyecto gestionado dentro de ella, formado por los requisitos, el modelo de dominio y las reglas de la aplicación por construir; [N3] es la aplicación generada. Todo requisito de este documento declara su nivel, y un requisito sin marca de nivel se considera mal formado.

La propiedad que gobierna la arquitectura se deriva de esa distinción: el dominio de la aplicación generada es dato para SLCP, no código. Incorporar un dominio nuevo no puede exigir modificación alguna del núcleo.

1.5 Sistema de identificación de los requisitos de este documento

Prefijo	Familia de requisitos
TRC	Trazabilidad y metamodelo
CON	Restricciones tecnológicas de construcción de la plataforma
TGT	Plataformas destino de la generación
NAM	Vocabulario canónico y convenciones de nomenclatura
INT	Interoperabilidad: exportación e importación
FUN	Funciones de la plataforma (SLCP-DOC-004)
ROL	Roles, membresía y autorización (SLCP-DOC-004)
REV	Ingeniería de ida y vuelta (SLCP-DOC-004)
ADM	Administración de la plataforma (SLCP-DOC-004, SLCP-DOC-005)
WBS	Estructura de desglose del trabajo (SLCP-DOC-005)
RES	Recursos y ponderación (SLCP-DOC-005)
PRG	Estado, avance y reapertura (SLCP-DOC-006)
MAP	Correspondencia requisito-trabajo (SLCP-DOC-006)
RQM	Gestión de requisitos (SLCP-DOC-007)
PLN	Secuencia planificación-ejecución (SLCP-DOC-007)
ANA	Análisis asistido de requisitos (SLCP-DOC-007)
RPT	Reportes (SLCP-DOC-007)
SPC	Especificación ejecutable (SLCP-DOC-008)
MOD	Modelado y diagramas (SLCP-DOC-008)
ARQ	Arquitectura y despliegue (SLCP-DOC-008, SLCP-DOC-009)
PRO	Edición y propagación de cambios (SLCP-DOC-009)
VER	Estrategia de prueba (SLCP-DOC-011)
Cada requisito lleva además su marca de nivel [N1], [N2] o [N3].	

2 Normalización terminológica aplicada

La terminología de este proyecto se ajusta al vocabulario normativo de Int [4]. La Tabla 1 registra las correcciones aplicadas respecto de formulaciones de uso corriente que inducen error de diseño.

Tabla 1: Correcciones terminológicas adoptadas

Formulación corriente	Término normativo	Motivo de la corrección
Ciclo de desarrollo del software	Procesos del ciclo de vida del software	El ciclo de vida excede al desarrollo: incluye operación, mantenimiento y retiro [2]
Requerimiento	Requisito	<i>Requirement</i> se traduce como requisito; <i>requerimiento</i> es un calco
Normas y estándares	Normas internacionales y estándares de la industria	En español técnico ambos términos son equivalentes; la distinción útil es entre norma formal y especificación de consorcio
Calidad (sin calificar)	Calidad de producto / calidad en uso / calidad de proceso	Modelos distintos y normas distintas [5, 6]
Tecnologías libres	Software libre o de código abierto con licencia aprobada por la OSI	<i>Libre</i> no significa gratuito ni de fuente visible [7]

Formulación corriente	Término normativo	Motivo de la corrección
Palabras reservadas (del metamodelo)	Vocabulario canónico controlado	Las palabras reservadas pertenecen a los lenguajes destino; el vocabulario canónico es del metamodelo
Testing	Verificación y validación	Son actividades distintas con objetos de traza distintos [8]

3 Marco normativo de referencia

Tabla 2: Normas y especificaciones aplicables

Referencia	Objeto en este proyecto
ISO/IEC/IEEE 12207:2017	Procesos del ciclo de vida del software orquestados por la plataforma [2]
ISO/IEC/IEEE 15288:2023	Procesos del ciclo de vida del sistema cuando el software es un elemento de un sistema mayor [9]
ISO/IEC/IEEE 29148:2018	Características de los requisitos y contenido de la ERS [1]
ISO/IEC 25010:2023	Modelo de calidad de producto; catálogo de atributos de calidad [5]
ISO/IEC 25019:2023	Modelo de calidad en uso, separado del anterior desde la segunda edición [6]
ISO/IEC/IEEE 42010:2022	Descripción de arquitectura y correspondencia vista-preocupación [10]
ISO/IEC/IEEE 29119-1/2/3	Conceptos, procesos y documentación de prueba [8, 11, 12]
ISO/IEC/IEEE 14764:2022	Mantenimiento y análisis de impacto [13]
ISO/IEC/IEEE 15289:2019	Contenido de los ítems de información del ciclo de vida [14]
ISO/IEC/IEEE 24765:2017	Vocabulario normativo (SEVOCAB) [4]
ISO/IEC 42001:2023	Gestión, registro y supervisión humana de la generación asistida por IA [15]
ISO/IEC 5055:2021	Medidas automatizadas de calidad del código fuente generado [16]
ISO/IEC 9075-1:2023	Marco del lenguaje SQL para el código de base de datos generado [17]
ISO/IEC/IEEE 29119-4:2021	Técnicas de diseño de prueba para derivar el oráculo de los artefactos generados [18]
WCAG 2.2	Criterios de accesibilidad de la interfaz [19]
ISO/IEC 25059:2023	Extensión del modelo de calidad aplicable por incorporar generación interpretativa [20]
ISO/IEC TR 29119-11:2020	Guías de prueba de sistemas basados en IA [21]
IEEE Std 828-2012	Contenido de la gestión de la configuración; estado <i>Inactive-Reserved</i> [22]

4 Modelo de trazabilidad

4.1 Ejes de trazabilidad

Int [1] distingue trazabilidad hacia atrás, hacia adelante y bidireccional. La plataforma debe implementar los tres, y además un cuarto eje que las normas clásicas no cubren y que resulta indispensable en una herramienta que produce artefactos de forma asistida: la trazabilidad de procedencia, es decir, el registro de qué motor, con qué insumos y bajo qué aprobación humana se produjo cada artefacto [15].

4.2 Metamodelo

TRC-01 Metamodelo explícito. [N1]

La plataforma debe implementar un metamodelo formal con, como mínimo, las entidades del vocabulario canónico de la Sección 7 y con relaciones tipadas, no con un enlace genérico sin semántica.

Verificación: El metamodelo es consultable como esquema formal legible por máquina y toda relación persistida válida contra él.

TRC-02 Relaciones tipadas. [N1]

El conjunto mínimo de relaciones debe ser: `satisfies`, `derivesFrom`, `refines`, `verifies`, `validates`, `implements`, `realizes`, `contributesTo`, `dependsOn`, `conflictsWith`, `supersedes`, `generatedBy` y `approvedBy`.

Verificación: Un enlace sin tipo o con tipo fuera del conjunto es rechazado por la capa de persistencia.

4.3 Identificación y versionado

TRC-03 Identificación estable. [N1][N2]

Cada instancia debe recibir un identificador que nunca se reutiliza ni se reasigna, compuesto por un UUID interno que actúa como clave de los enlaces y un identificador legible que actúa solo como presentación.

Verificación: Al cambiar el dominio funcional de un elemento, su identificador legible cambia y ningún enlace se rompe.

TRC-04 Versionado inmutable. [N1][N2]

Toda modificación de contenido debe generar una nueva versión con marca temporal y autor, nunca una sobreescritura.

Verificación: El histórico completo de cualquier elemento es recuperable y ninguna operación de la interfaz permite eliminarlo.

4.4 Calidad de los requisitos como precondition

TRC-05 Características obligatorias. [N2]

La plataforma debe evaluar cada requisito contra las características de Int [1] y marcar como no conforme el que las incumpla, con atención especial a la singularidad: un requisito que

enuncia dos necesidades no es trazable, porque un enlace no puede indicar cuál de las dos quedó implementada.

Verificación: Existe un informe de conformidad por requisito y la línea base no admite requisitos no conformes.

TRC-06 Metadatos mínimos. [N2]

Cada requisito debe portar criterio de aceptación verificable, prioridad y origen; si es un requisito de calidad, debe portar además la característica de Int [5] a la que pertenece y su medida asociada.

Verificación: Ningún requisito sin criterio de aceptación puede pasar a línea base; la herramienta lo bloquea.

4.5 Cadena de enlaces

TRC-07 Enlace hacia el origen. [N2]

Todo requisito debe enlazar con al menos una entidad de origen: parte interesada identificada, necesidad de negocio, requisito de nivel superior o cláusula regulatoria.

Verificación: El indicador de requisitos huérfanos es cero antes de aprobar la línea base de la ERS.

TRC-08 Enlace a decisiones de arquitectura. [N2]

Cada decisión de arquitectura debe registrar los requisitos que la motivan y los que quedan comprometidos por ella, materializando como enlace navegable la correspondencia entre vistas y preocupaciones que exige Int [10].

*Verificación: Cada registro de decisión tiene al menos un enlace *satisfies* y, si aplica, enlaces *conflictsWith* hacia los requisitos sacrificados.*

TRC-09 Enlace de los atributos de calidad. [N2]

Los requisitos de calidad deben trazar específicamente a los mecanismos arquitectónicos que los realizan, y no únicamente al módulo funcional donde se manifiestan.

Verificación: Ningún requisito de calidad queda enlazado solo a documentación descriptiva.

TRC-10 Enlace a la construcción. [N2][N3]

En el repositorio del proyecto gestionado, el vínculo entre requisito y código debe capturarse automáticamente desde el sistema de control de versiones mediante convención obligatoria de mensaje de *commit*, validada por un *hook* que rechaza el *commit* no conforme.

Verificación: Un commit sin identificador de requisito válido es rechazado, y el enlace registra el hash como evidencia inmutable.

TRC-11 Detección de artefactos huérfanos. [N2][N3]

Todo artefacto de código sin requisito asociado debe reportarse como huérfano, por ser indicador de alcance no autorizado. El cómputo comprende los bloques protegidos y los artefactos anulados, conforme a PRO-23.

Verificación: El informe de huérfanos se genera en cada integración y su valor distinto de cero bloquea la promoción a línea base.

TRC-12 Enlace a verificación. [N2]

Cada requisito debe tener al menos un caso de prueba que lo verifique, y cada ejecución debe registrar caso, versión del requisito, versión del elemento probado, entorno, resultado y evidencia, de acuerdo con las plantillas de Int [12].

Verificación: Las estructuras de datos de prueba son isomorfas a dichas plantillas y su exportación es directa.

TRC-13 Separación de verificación y validación. [N2]

La validación debe trazar contra la entidad de necesidad y no contra el requisito, porque responde a una pregunta distinta.

*Verificación: Existen dos tipos de enlace diferenciados, *verifies* y *validates*, con extremos de tipo distinto.*

4.6 Cobertura, cambio e impacto**TRC-14 Métricas de cobertura. [N2]**

La plataforma debe calcular de forma continua la cobertura hacia adelante, la cobertura hacia atrás, los requisitos huérfanos, los artefactos huérfanos, los enlaces sospechosos y los requisitos sin verificación ejecutada.

Verificación: Las seis métricas son consultables por API y su valor se registra históricamente por línea base.

TRC-15 Líneas base. [N2]

La plataforma debe permitir congelar líneas base identificadas y firmadas del conjunto de requisitos y de la matriz de trazabilidad del proyecto gestionado.

Verificación: Una línea base cerrada no admite modificación; toda variación posterior genera una nueva línea base.

TRC-16 Enlaces sospechosos. [N2]

Cuando un elemento cambia de versión, todos los enlaces que lo tocan deben marcarse automáticamente como sospechosos y propagar ese estado a los elementos dependientes hasta que un responsable humano los revalide de forma explícita.

Verificación: La revalidación queda registrada con identidad, fecha y versión exacta revisada.

TRC-17 Análisis de impacto. [N2]

Toda petición de cambio debe producir un informe de impacto generado desde el grafo de trazabilidad, conforme al análisis de impacto de Int [13].

Verificación: El informe enumera requisitos, decisiones, unidades de código y casos de prueba afectados, y se genera sin intervención manual.

TRC-18 Trazabilidad de defectos. [N2]

Todo defecto debe enlazar con el requisito incumplido o registrarse como requisito faltante, y toda corrección urgente debe trazar a defecto, requisito y prueba de regresión añadida.

Verificación: No existen defectos cerrados sin prueba de regresión enlazada.

TRC-19 Clasificación del mantenimiento. [N2]

Toda petición de cambio debe clasificarse como correctiva, adaptativa, perfectiva o preventiva según Int [13].

Verificación: El campo es obligatorio y su ausencia impide el flujo de aprobación.

TRC-20 Trazabilidad de liberación. [N2][N3]

Cada liberación debe responder automáticamente qué requisitos incluye, en qué versión, verificados por qué ejecuciones y sobre qué *commits*.

Verificación: El informe de liberación se genera desde el grafo y se acompaña del inventario de componentes de la Sección 8.

4.7 Procedencia de los artefactos generados**TRC-21 Registro de procedencia. [N1]**

Todo artefacto producido de forma automatizada debe registrar insumos exactos con identificador y versión, plantilla o instrucción aplicada, identificador y versión del motor generador, marca temporal y estado de revisión.

Verificación: El registro de procedencia es obligatorio en el esquema y su ausencia impide persistir el artefacto.

TRC-22 Aprobación humana. [N1]

Ningún artefacto generado puede pasar a línea base sin aprobación humana registrada con identidad, fecha y versión exacta aprobada; hasta ese momento debe tratarse como propuesta [15].

Verificación: El estado `PROPOSED` no es promovible sin un enlace `approvedBy`.

TRC-23 Declaración del régimen de determinismo. [N1][N3]

Cada artefacto generado debe declarar de forma explícita su régimen de determinismo. Los artefactos producidos por plantilla determinista deben ser reproducibles a partir de sus insumos registrados. Los artefactos producidos por generación interpretativa no pueden prometer reproducibilidad; deben declararlo, registrar los parámetros que permitirían aproximar su reproducción, y sustituir la garantía de derivación por evidencia de verificación conforme a TGT-08 y siguientes.

Verificación: Cada artefacto expone su régimen declarado, y ningún artefacto de régimen interpretativo alcanza línea base sin la evidencia de verificación exigida.

4.8 Persistencia y auditoría**TRC-24 Almacén de solo anexo. [N1]**

El almacén de trazabilidad de la plataforma debe ser de solo anexo: las correcciones se registran como nuevos eventos, nunca como borrado o edición del histórico.

Verificación: No existe operación de la API que elimine un evento; la auditoría lo comprueba con una prueba negativa automática. La retirada del servicio de un proyecto o de una cuenta se realiza mediante baja, que es un evento adicional y no una eliminación, según SLCP-ADR-0003; la única operación que destruye datos es la

purga de datos personales de ADM-08, sujeta a fundamento jurídico y doble autorización.

TRC-25 Contenido documental normalizado. [N2]

El contenido de los ítems de información debe ajustarse a Int [14].

Verificación: Cada plantilla de documento declara la cláusula de 15289 que la fundamenta.

5 Restricciones tecnológicas de construcción

5.1 Definición operativa de libre

CON-01 Criterio de licencia. [N1]

La plataforma SLCP se distribuye bajo licencia MIT. Todo componente incorporado a ella debe distribuirse bajo una licencia que cumpla la Open Source Definition [7], sea compatible con MIT, y debe declararse mediante su identificador normalizado de la SPDX License List [23].

Verificación: El inventario de componentes no contiene ningún identificador ausente de la lista de licencias aprobadas del proyecto.

CON-02 Prohibición de dependencias propietarias en construcción. [N1]

La compilación, la ejecución de pruebas y el empaquetado de la plataforma no deben requerir ningún artefacto de licencia propietaria, ni siquiera de descarga gratuita.

Verificación: Una construcción limpia en un contenedor sin credenciales ni repositorios privados finaliza correctamente.

5.2 Delimitación entre plataforma y destino

Distinción crítica. La restricción de software libre aplica a la plataforma SLCP, no a las plataformas destino. SLCP debe poder generar código para Oracle Database o para .NET sin incorporar artefactos propietarios en su propia construcción. Sin esta distinción, la restricción de construcción libre y el requisito de generar para destinos propietarios se contradicen entre sí.

CON-03 Independencia respecto de artefactos propietarios del destino. [N1][N3]

La generación de código para un destino propietario debe producirse a partir de plantillas y metadatos propios, sin enlazar ni redistribuir controladores, bibliotecas ni SDK propietarios; cuando la ejecución del artefacto generado requiera tales componentes, la plataforma debe documentar la dependencia y dejar su obtención a cargo de la persona usuaria.

Verificación: La generación para Oracle y para .NET se completa en un entorno que no contiene ningún artefacto propietario.

5.3 Higiene de licencias

CON-04 Aislamiento del copyleft fuerte. [N1]

Los componentes bajo copyleft fuerte deben invocarse a través de una frontera de proceso o servicio y no enlazarse en el mismo binario, salvo que la licencia elegida para SLCP sea compatible con esa incorporación.

Verificación: El análisis de composición de software no reporta enlaces incompatibles con la licencia declarada de SLCP.

CON-05 Inventario de componentes. [N1]

Cada versión de la plataforma debe publicar un inventario de componentes en formato SPDX [24] y CycloneDX [25].

Verificación: Ambos archivos se generan en la canalización de integración y validan contra sus esquemas respectivos.

CON-06 Licencia del proyecto generado. [N3]

La plataforma debe permitir declarar la licencia de la aplicación generada, propagarla a sus artefactos y emitir su inventario de componentes, sin imponer ninguna restricción sobre cuál sea esa licencia. Las restricciones CON-01 a CON-05 aplican únicamente a la plataforma y no se trasladan a la aplicación generada, cuya elección corresponde a quien la encarga.

Verificación: La generación con una licencia propietaria declarada se completa sin advertencia ni bloqueo, y el inventario emitido refleja la licencia declarada.

5.4 Pila tecnológica de construcción

La Tabla 3 recoge la pila que satisface CON-01 a CON-05. Las capas de interfaz y de servicio quedaron fijadas como decisión firme en SLCP-DOC-004: Angular para la interfaz y Java con Spring Boot para el servicio. Las restantes capas siguen siendo propuesta de arquitectura, y cualquier sustitución es admisible en ellas si conserva el criterio de licencia y las capacidades indicadas.

Tabla 3: Pila tecnológica propuesta para la construcción de SLCP

Capa	Opción propuesta	Justificación
Lenguaje del núcleo	Java LTS sobre una distribución abierta de OpenJDK	Ecosistema maduro de modelado, plantillas y análisis estático; coincide con uno de los destinos, lo que reduce la distancia conceptual
Framework de servicio	Spring Boot (Apache-2.0) — decidido	Decisión firme registrada en SLCP-DOC-004; construcción libre y despliegue en contenedor
Persistencia relacional	PostgreSQL	Licencia permisiva y soporte nativo de JSON, necesario para el registro de procedencia

Capa	Opción propuesta	Justificación
Grafo de trazabilidad	Extensión de grafo sobre PostgreSQL (Apache AGE)	Evita introducir un motor bajo copyleft fuerte y mantiene una única fuente de verdad transaccional
Motor de plantillas	FreeMarker o Velocity (Apache-2.0)	Separación estricta entre plantilla y modelo, condición de TGT-05
Modelado y transformación	Eclipse EMF, Xtext, Acceleo (EPL)	Soporte nativo de metamodelos y de generación dirigida por modelos
Intercambio de requisitos	Bibliotecas ReqIF de Eclipse RMF (EPL)	Implementación de referencia del formato de Obj [26]
Frontend	Angular (MIT) — decidido	Decisión firme registrada en SLCP-DOC-004; licencia permisiva compatible con CON-01
Identidad y acceso	Keycloak (Apache-2.0)	Federación y registro de identidad para las aprobaciones de TRC-22
Integración continua	Forgejo, GitLab CE o Jenkins	Ejecución autoalojada sin dependencia de servicios propietarios
Contenedores	Podman o Docker Engine	Reproducibilidad de la construcción exigida por CON-02
Documentación	Asciidoctor, Pandoc y $\LaTeX$	Cadena completa hacia PDF/A sin componentes propietarios
Diagramación	PlantUML o Graphviz invocados como proceso externo	Cumple CON-04 al aislar el copyleft tras una frontera de proceso

6 Plataformas destino de la generación

6.1 Criterio de selección

Los destinos se seleccionan por difusión efectiva en la industria y no por preferencia técnica. La encuesta de Stack Overflow [27] y el ranking mensual de DB-Engines [28] constituyen la evidencia pública utilizada; ambos miden popularidad declarada o interés en línea, no instalaciones reales, y esa limitación debe tenerse presente al revisar esta sección.

6.2 Lenguajes

TGT-01 Lenguajes destino obligatorios. [N3]

La plataforma debe generar código para Java, C# y PHP como conjunto mínimo, y debería admitir la incorporación posterior de TypeScript y Python sin modificar el núcleo.

Verificación: Existe al menos un generador completo y probado por cada lenguaje obligatorio, y la incorporación de un lenguaje nuevo no requiere cambios fuera del módulo de plantillas.

6.3 Sistemas gestores de bases de datos

TGT-02 SGBD destino obligatorios. [N3]

La plataforma debe generar esquemas y código de acceso para PostgreSQL, MySQL, MariaDB y Oracle Database, y debería admitir Microsoft SQL Server.

Verificación: Un mismo modelo de datos genera esquemas válidos y ejecutables en los cuatro destinos obligatorios.

TGT-03 SQL conforme y dialectos aislados. [N3]

El código SQL generado debe ajustarse a Int [17] en su núcleo, y toda construcción específica de dialecto debe residir en un módulo de dialecto identificado y sustituible.

Verificación: El núcleo generado se ejecuta sin cambios en los cuatro destinos; las diferencias residen exclusivamente en el módulo de dialecto.

TGT-04 Migraciones versionadas. [N3]

Todo cambio de esquema debe generarse como migración versionada y reversible, enlazada al requisito que la motiva.

Verificación: Cada migración tiene enlace `implements` hacia un requisito y una operación de reversión probada.

6.4 Frameworks

Destino	Backend	Acceso a datos
Java	Spring Boot; Jakarta EE mediante Quarkus	JPA con Hibernate; Flyway o Liquibase
C#	ASP.NET Core	Entity Framework Core con sus migraciones
PHP	Laravel; Symfony	Doctrine ORM y Doctrine Migrations
Frontend: React, Angular, Vue o Svelte, con contrato de API descrito en OpenAPI [29]		

TGT-05 Separación entre núcleo y plantillas. [N1]

El núcleo de la plataforma no debe contener conocimiento de ningún lenguaje, SGBD ni framework destino; todo ese conocimiento debe residir en plantillas y descriptores de destino declarativos.

Verificación: Añadir un framework destino nuevo se realiza añadiendo plantillas y un descriptor, sin recompilar ni modificar el núcleo.

TGT-06 Calidad medible del código generado. [N3]

El código generado debe evaluarse con medidas automatizadas conforme a Int [16] antes de entregarse.

Verificación: Ningún artefacto se entrega con debilidades de las categorías de seguridad y fiabilidad por encima del umbral definido por el proyecto.

TGT-07 Contrato de API primero. [N3]

Para cada servicio generado debe producirse su descripción OpenAPI [29] como artefacto de

primer nivel, enlazado a los requisitos que realiza.

Verificación: La descripción valida contra el esquema de OpenAPI y el código generado es coherente con ella.

6.5 Generación de la lógica de negocio

La plataforma debe generar el código fuente de la lógica de negocio, según lo establecido en SLCP-ADR-0002. Los requisitos siguientes definen el régimen de verificación que hace esa generación auditable.

TGT-08 Oráculo antes que implementación. [N2][N3]

Antes de generar la implementación de un elemento de lógica de negocio, la plataforma debe derivar de los criterios de aceptación del requisito el conjunto de casos de prueba que definirá su corrección, aplicando las técnicas de Int [18], y debe sellarlo con huella criptográfica.

Verificación: El oráculo existe y está sellado antes del primer artefacto de implementación; toda alteración posterior exige aprobación humana independiente registrada como evento distinto.

TGT-09 Especificación suficiente como precondition. [N2]

Si un requisito carece de criterio de aceptación verificable, la plataforma no debe generar su lógica; debe solicitar la información faltante e identificar el requisito como incompleto.

Verificación: Un requisito sin criterio de aceptación produce una solicitud de información, nunca código.

TGT-10 Verificación en capas con bloqueo. [N3]

La promoción de un artefacto de lógica de negocio debe exigir, en orden: compilación limpia, ejecución en verde del oráculo sellado, comprobación de las invariantes declaradas, medidas automatizadas de Int [16] sin debilidades de seguridad ni de fiabilidad, y cobertura mínima definida por el proyecto.

Verificación: El fallo de cualquier capa detiene la promoción y queda registrado con su causa.

TGT-11 Escalado por agotamiento. [N1]

Si tras el número de iteraciones configurado el oráculo no se satisface, o si la complejidad del artefacto supera el umbral definido, la plataforma debe detener la generación y entregar la interfaz, el oráculo y un informe de bloqueo, marcando el elemento para desarrollo humano.

Verificación: El agotamiento produce un resultado explícito y auditable, nunca un artefacto entregado sin verificación superada.

TGT-12 Regeneración por propuesta. [N3]

Una implementación aprobada queda fijada y versionada. Una regeneración posterior no debe sobrescribirla: debe producir una propuesta acompañada de la diferencia respecto de la versión vigente y del resultado del oráculo sobre ambas, y su sustitución exige aprobación expresa. Este requisito queda subordinado a la precedencia de verdad de PRO-15: la regeneración nunca produce un artefacto que contradiga una especificación humana vigente.

Verificación: No existe ninguna ruta que reemplace código aprobado sin decisión humana registrada.

TGT-13 Gobierno de la generación. [N1]

La plataforma debe medir y publicar la tasa de aceptación en el primer intento, el número medio de iteraciones hasta el verde, la tasa de escalado a desarrollo humano y la densidad de defectos posteriores por artefacto generado.

Verificación: Las cuatro métricas son consultables por API y se registran históricamente por línea base.

TGT-14 Independencia del dominio. [N1]

El núcleo de la plataforma no debe contener conocimiento de ningún dominio de aplicación. El dominio de la aplicación generada es dato del proyecto gestionado, nunca código del núcleo. Es la extensión de TGT-05, que ya prohibía el conocimiento de lenguajes, gestores de bases de datos y frameworks destino.

Verificación: Dar soporte a un dominio nuevo se realiza cargando un proyecto gestionado, sin recompilar ni modificar el núcleo, y una búsqueda léxica del código del núcleo no encuentra ningún término de dominio de los casos de prueba.

7 Vocabulario canónico y convenciones de nomenclatura

7.1 Regla de idioma

NAM-01 Identificadores en inglés. [N1][N3]

Todo identificador técnico de la plataforma — tipos del metamodelo, atributos, relaciones, claves de serialización, objetos de base de datos, rutas de API, ramas y nombres de archivo — debe expresarse en inglés. En la aplicación generada la regla alcanza únicamente a los identificadores estructurales que la plataforma introduce por su cuenta; los identificadores derivados del dominio conservan el idioma del glosario del proyecto gestionado.

Verificación: Un análisis léxico del código y del esquema de la plataforma no encuentra identificadores en español fuera de los archivos de traducción, y ningún término de dominio aparece traducido en el código generado.

NAM-02 Separación entre identificador y etiqueta. [N1][N3]

Cada tipo y cada atributo debe tener un identificador canónico invariable en inglés y una etiqueta traducible independiente.

Verificación: Cambiar el idioma de la interfaz no altera ningún identificador persistido ni ninguna clave de serialización.

7.2 Vocabulario canónico

Tabla 4: Vocabulario canónico controlado

Concepto (es)	Identificador canónico	Prefijo de identificador
Cuenta de usuario	User	USR
Proyecto	Project	PRJ
Equipo del proyecto	ProjectTeam	PTM
Membresía en el proyecto	ProjectMembership	PMB
Aprobación	Approval	APV

Concepto (es)	Identificador canónico	Prefijo
Invitación	Invitation	INV
Elemento de trabajo	WorkItem	WIT
Recurso	Resource	RSC
Asignación de recurso	ResourceAssignment	RAS
Cambio de estado	StatusChange	STC
Hallazgo de análisis	AnalysisFinding	FND
Definición de reporte	ReportDefinition	RPD
Escenario de aceptación	Scenario	SCN
Caso de uso	UseCase	UCS
Componente de software	SoftwareComponent	SWC
Vista de arquitectura	ArchitectureView	AVW
Nodo de despliegue	DeploymentNode	DPN
Punto de vista	ArchitectureViewpoint	AVP
Anulación declarada	DeclaredOverride	OVR
Bloque protegido	ProtectedBlock	PBK
Ejecución de pruebas	TestRun	RUN
Resultado de prueba	TestResult	TRS
Instrumento de verificación	VerificationInstrument	INS
Respuesta al instrumento	InstrumentResponse	IRP
Parte interesada	Stakeholder	STK
Necesidad	StakeholderNeed	NED
Requisito	Requirement	REQ
Atributo de calidad	QualityAttribute	QUA
Decisión de arquitectura	ArchitectureDecision	ADR
Elemento de diseño	DesignElement	DSN
Unidad de código	CodeUnit	CDU
Caso de prueba	TestCase	TST
Ejecución de prueba	TestExecution	EXE
Defecto	Defect	DEF
Petición de cambio	ChangeRequest	CHG
Liberación	Release	REL
Regla de negocio	BusinessRule	RUL
Oráculo de prueba	TestOracle	ORC
Artefacto generado	GeneratedArtifact	GEN
Enlace de trazabilidad	TraceLink	TRL
Línea base	Baseline	BSL
Elemento de configuración	ConfigurationItem	CFG
Ítem de información	InformationItem	INF

NAM-03 Vocabulario cerrado. [N1]

Los identificadores de la Tabla 4 constituyen un vocabulario cerrado: ninguna capa puede introducir un sinónimo ni una variante ortográfica del mismo concepto.

Verificación: Una prueba automática de coherencia léxica falla ante cualquier sinónimo no declarado.

7.3 Convenciones por artefacto

Tabla 5: Convenciones de nomenclatura por artefacto

Ámbito	Convención	Ejemplo
Tipos del metamodelo	UpperCamelCase, sustantivo singular	ChangeRequest
Atributos y relaciones	lowerCamelCase	approvedBy
Claves JSON y JSON-LD	lowerCamelCase	qualityAttribute
Literales de enumeración	UPPER_SNAKE_CASE	PROPOSED
Tablas y columnas SQL	snake_case en minúsculas; tabla en plural, column en singular	change_requests, approved_by
Restricciones SQL	prefijo de tipo y nombres de tabla y columna	fk_requirements_baseline_id
Java: paquetes	minúsculas, dominio invertido [30]	org.slcp.traceability
Java: clases y métodos	UpperCamelCase y lowerCamelCase [31]	TraceLinkService.findByRequirement
C#: tipos, métodos y propiedades	PascalCase; parámetros en camelCase; interfaces con prefijo I [32]	ITraceLinkRepository.FindByRequirement
PHP: clases y métodos	StudlyCaps y camelCase; espacios de nombres según PSR-4 [33, 34]	Slcp\Traceability\TraceLinkService
Constantes en los tres lenguajes	UPPER_SNAKE_CASE	MAX_TRACE_DEPTH
Rutas REST	kebab-case en plural	/api/v1/change-requests
Archivos de frontend	kebab-case	trace-link-graph.tsx
Ramas de Git	prefijo de tipo e identificador de requisito	feat/REQ-AUTH-0017-token-refresh

NAM-04 Transformación determinista. [N1][N3]

La conversión entre la forma canónica y la forma de cada destino debe realizarla una función única y determinista de la plataforma, nunca la plantilla individual.

Verificación: Dado un identificador canónico, la forma generada para cada destino es idéntica en todas las plantillas.

7.4 Colisión con palabras reservadas**NAM-05 Prohibición de colisión. [N3]**

Ningún identificador canónico ni ninguna forma derivada puede coincidir con una palabra reservada de SQL [17], Java, C# o PHP, ni con una palabra reservada de los dialectos de los SGBD destino.

Verificación: El generador consulta una lista de exclusión por destino y aborta con error explícito ante cualquier colisión, en lugar de entrecomillar el identificador de forma silenciosa.

NAM-06 Límite de longitud portable. [N3]

Los identificadores de base de datos generados deben mantenerse dentro del límite más restrictivo de los SGBD destino habilitados en el proyecto, y el generador debe aplicar una regla de abreviatura determinista y reversible cuando lo exceda.

Verificación: Ningún identificador generado es truncado de forma no determinista ni produce colisión tras la abreviatura.

NAM-07 Glosario de dominio. [N2]

Cada proyecto gestionado debe mantener un glosario con un término canónico por concepto de dominio, en el idioma que el proyecto determine, del cual se derivan de forma determinista todos los identificadores de la aplicación generada.

Verificación: Un concepto de dominio produce siempre el mismo identificador en todos los artefactos generados, y ningún concepto tiene dos términos canónicos.

NAM-08 Validación del nombre propuesto. [N1][N3]

Cuando una persona proponga un nombre, la plataforma debe comprobar si cada destino habilitado lo admite. Si todos lo admiten, debe aceptarlo. Si alguno no lo admite, debe indicar cuál y por qué, ofrecer alternativas legales y exigir que la persona elija una.

Verificación: Ningún nombre legal se rechaza y ningún nombre ilegal se sustituye sin elección humana.

NAM-09 Registro de la elección en el glosario. [N1][N2]

La alternativa elegida debe registrarse en el glosario del proyecto, de modo que se aplique de forma idéntica en todos los artefactos y destinos a partir de ese momento.

Verificación: La misma colisión no vuelve a plantearse en otro artefacto.

NAM-10 Sin sustitución ni entrecomillado silencioso. [N1][N3]

La plataforma no debe sustituir un nombre por su cuenta ni recurrir al entrecomillado del identificador para sortear la colisión, por perderse con ello la portabilidad entre dialectos.

Verificación: Toda resolución de colisión procede de una elección humana registrada.

7.5 Esquema de identificadores legibles

Formato:	<PREFIX>-<DOMAIN>-<SEQUENCE>-v<VERSION>
Ejemplos:	REQ-AUTH-0017-v3 ADR-SEC-0004-v1 TST-AUTH-0017-02-v1
Reglas:	PREFIX proviene de la Tabla de vocabulario canonico DOMAIN codigo de dominio funcional, en ingles y en mayusculas SEQUENCE entero con relleno de ceros, no reutilizable VERSION entero incremental, obligatorio en toda referencia

8 Interoperabilidad: exportación e importación

8.1 Principios

INT-01 Formato abierto por defecto. [N2]

Todo artefacto debe poder exportarse al menos a un formato abierto y documentado, además del formato nativo del proyecto.

Verificación: No existe ningún tipo de artefacto cuyo único medio de salida sea el formato nativo.

INT-02 Simetría de importación. [N2]

Todo formato de exportación que represente información estructurada debe tener su correspondiente importador.

Verificación: La matriz de formatos no presenta ninguna casilla de exportación sin importación, salvo los formatos de presentación declarados como terminales.

8.2 Formatos por tipo de artefacto

Tabla 6: Formatos de intercambio por tipo de artefacto

Artefacto	Formatos	Dirección
Requisitos	ReqIF 1.2 [26]; CSV [35]; JSON [36]; OOXML [37]; ODF [38]	Entrada y salida
Trazabilidad	OSLC 3.0 como API [39]; JSON-LD [40]; CSV de matriz	Entrada y salida
Arquitectura y diseño	XMI [41] sobre UML [42]; PlantUML; Mermaid; SVG	Entrada y salida, salvo SVG
Procesos de negocio	BPMN 2.0 [43]	Entrada y salida
Reglas de decisión	DMN [44]	Entrada y salida
Modelo de datos	DDL SQL [17]; migraciones declarativas en XML o YAML	Entrada y salida
Contratos de servicio	OpenAPI [29]; GraphQL SDL	Entrada y salida
Pruebas	Plantillas de 29119-3 [12]; Gherkin [45]; JUnit XML de resultados	Entrada y salida
Componentes y licencias	SPDX [24]; CycloneDX [25]	Salida
Documentación	Markdown; AsciiDoc; $\LaTeX$; OOXML; ODF; PDF 2.0 [46] y PDF/A-4 [47]	Salida; entrada en los formatos de texto

8.3 Requisitos de importación

INT-03 Validación previa. [N2]

Todo artefacto importado debe validarse contra el esquema de su formato antes de persistirse, y el rechazo debe señalar la ubicación exacta del incumplimiento.

Verificación: Un archivo malformado no produce ninguna escritura parcial en el almacén.

INT-04 Simulación previa. [N2]

Toda importación debe ofrecer un modo de simulación que informe qué se crearía, qué se actualizaría y qué entraría en conflicto, sin modificar el estado.

Verificación: La simulación y la ejecución real producen el mismo informe cuando no hay cambios concurrentes.

INT-05 Correspondencia de identificadores. [N2]

La importación debe preservar el identificador de origen en un campo dedicado y generar identificadores propios según NAM, sin sobrescribir identificadores existentes.

Verificación: Ningún identificador nativo es reasignado durante una importación.

INT-06 Procedencia del artefacto importado. [N2]

Cada elemento importado debe registrar herramienta de origen, formato, versión del formato, nombre y huella criptográfica del archivo, y responsable de la importación.

Verificación: El registro de procedencia permite reconstruir el origen de cualquier elemento importado.

INT-07 Resolución explícita de conflictos. [N2]

Ante un conflicto la plataforma debe requerir decisión humana entre conservar, reemplazar o bifurcar, y registrar la decisión adoptada.

Verificación: No existe ninguna ruta de código que resuelva conflictos de forma automática y silenciosa.

INT-08 Informe de pérdida. [N2]

Cuando un formato de destino no pueda representar parte de la información, la exportación debe emitir un informe de pérdida que enumere los elementos no representados.

Verificación: La exportación a un formato de menor expresividad siempre va acompañada de dicho informe.

8.4 Ida y vuelta y formato nativo**INT-09 Ida y vuelta sin pérdida. [N2]**

Exportar un proyecto al formato nativo y volver a importarlo debe producir un estado semánticamente idéntico al original, incluidos enlaces, versiones y registros de procedencia.

Verificación: La comparación normalizada entre el estado original y el reimportado no arroja diferencias.

INT-10 Paquete de proyecto. [N2]

El formato nativo debe ser un paquete comprimido con manifiesto versionado, contenido serializado en JSON-LD [40] y huella criptográfica por archivo.

Verificación: El manifiesto declara la versión del esquema y la verificación de huellas detecta cualquier alteración.

INT-11 Compatibilidad de versiones del esquema. [N2]

La plataforma debe importar paquetes generados por versiones anteriores del esquema mediante migraciones declaradas, y rechazar de forma explícita los generados por versiones posteriores.

Verificación: Existe una prueba de importación por cada versión histórica del esquema soportada.

9 Criterios de aceptación consolidados

Los siguientes criterios son condiciones de piso: su incumplimiento invalida la entrega, con independencia del grado de avance funcional.

1. La construcción completa de la plataforma finaliza en un entorno limpio sin ningún artefacto propietario (CON-02).
2. El inventario de componentes se genera en SPDX y CycloneDX y no contiene licencias fuera de la lista aprobada (CON-01, CON-05).
3. Ningún requisito alcanza línea base sin criterio de aceptación, origen enlazado y al menos un caso de prueba (TRC-06, TRC-07, TRC-12).
4. Las seis métricas de trazabilidad son consultables y sus valores de bloqueo son cero antes de cada liberación (TRC-14).
5. Ningún artefacto generado alcanza línea base sin aprobación humana registrada (TRC-22).
6. Ningún identificador en español aparece en el código de la plataforma fuera de los archivos de traducción, y ningún término de dominio aparece traducido en el código generado (NAM-01, NAM-07).
7. Dar soporte a un dominio nuevo no requiere recompilar el núcleo (TGT-14).
8. Ningún artefacto es aprobado por su propio autor, y ninguna persona reúne los roles de desarrollador y propietario del producto en el mismo proyecto (ROL-04, ROL-06).
9. El generador aborta ante colisión con palabra reservada en lugar de entrecomillar de forma silenciosa (NAM-05).
10. Ningún artefacto de lógica de negocio alcanza línea base sin oráculo sellado previo, ejecución en verde y revisión humana con constancia por artefacto (TGT-08, TGT-10, TRC-22).
11. El agotamiento de iteraciones produce informe de bloqueo y nunca un artefacto promovido sin verificación (TGT-11).
12. La ida y vuelta al formato nativo no produce diferencias semánticas (INT-09).

10 Asuntos abiertos

1. **Licencia de SLCP.** La elección entre una licencia permisiva y una copyleft condiciona directamente CON-04 y debe decidirse antes de fijar la pila definitiva.
2. **Alcance de la generación. Cerrado.** Resuelto por SLCP-ADR-0002: la plataforma genera la totalidad de los artefactos del ciclo de vida, incluido el código fuente de la lógica de negocio.
3. **Estrategia de reconciliación. Cerrado.** Resuelto por TGT-12: la regeneración produce propuesta con diferencia y nunca sobreescribe código aprobado.
4. **Umbral operativo.** Deben fijarse el número máximo de iteraciones de TGT-11, el umbral de complejidad de escalado y la cobertura mínima de TGT-10; son parámetros de calibración que requieren datos del primer caso semilla.

5. **Riesgos residuales bajo vigilancia.** Los riesgos R-01 a R-05 de SLCP-ADR-0002 no quedan eliminados por el diseño y requieren revisión en cada iteración.
6. **Edición aplicable de 12207.** Debe verificarse el estado de publicación de la segunda edición antes de emitir cualquier declaración de conformidad [3].

A Palabras reservadas: fuentes de la lista de exclusión

La lista de exclusión exigida por NAM-05 debe construirse a partir de las siguientes fuentes y versionarse junto con el descriptor de cada destino: las palabras reservadas de la serie ISO/IEC 9075 [17]; las palabras reservadas específicas de PostgreSQL, MySQL, MariaDB y Oracle Database publicadas en la documentación de cada producto; las palabras clave de Java, C# y PHP publicadas en sus especificaciones de lenguaje respectivas. La lista debe revisarse en cada actualización de versión mayor de cualquier destino habilitado, porque los conjuntos de palabras reservadas se amplían entre versiones.

Referencias

- [1] *ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering*. International Organization for Standardization, Geneva, Switzerland, 2018. <https://www.iso.org/standard/72089.html>.
- [2] *ISO/IEC/IEEE 12207:2017 Systems and software engineering — Software life cycle processes*. International Organization for Standardization, Geneva, Switzerland, 2017. Edición 1, 2017-11; revisada y confirmada en 2023. <https://www.iso.org/standard/63712.html>.
- [3] *ISO/IEC/IEEE 12207 systems and software engineering — software life cycle processes, edition 2*. ISO, estado *Under publication* (Edición 2, 2026-04); reemplazará a la edición de 2017, 2026. <https://committee.iso.org/standard/90219.html>.
- [4] *ISO/IEC/IEEE 24765:2017 Systems and software engineering — Vocabulary*. International Organization for Standardization, Geneva, Switzerland, 2017. Base terminológica SEVO-CAB. <https://www.iso.org/standard/71952.html>.
- [5] *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. International Organization for Standardization, Geneva, Switzerland, 2023. Edición 2, 2023-11; cancela y reemplaza a ISO/IEC 25010:2011 junto con ISO/IEC 25002 e ISO/IEC 25019. <https://www.iso.org/standard/78176.html>.
- [6] *ISO/IEC 25019:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality-in-use model*. International Organization for Standardization, Geneva, Switzerland, 2023. <https://www.iso.org/standard/78177.html>.
- [7] *The Open Source Definition*. Open Source Initiative, 2024. <https://opensource.org/osd>.
- [8] *ISO/IEC/IEEE 29119-1:2022 Software and systems engineering — Software testing — Part 1: General concepts*. International Organization for Standardization, Geneva, Switzerland, 2022. <https://www.iso.org/standard/81291.html>.
- [9] *ISO/IEC/IEEE 15288:2023 Systems and software engineering — System life cycle processes*. International Organization for Standardization, Geneva, Switzerland, 2023. Edición 2, 2023-05. <https://www.iso.org/standard/81702.html>.
- [10] *ISO/IEC/IEEE 42010:2022 Software, systems and enterprise — Architecture description*. International Organization for Standardization, Geneva, Switzerland, 2022. <https://www.iso.org/standard/74393.html>.
- [11] *ISO/IEC/IEEE 29119-2:2021 Software and systems engineering — Software testing — Part 2: Test processes*. International Organization for Standardization, Geneva, Switzerland, 2021. Edición 2, 2021-10. <https://www.iso.org/standard/79428.html>.
- [12] *ISO/IEC/IEEE 29119-3:2021 Software and systems engineering — Software testing — Part 3: Test documentation*. International Organization for Standardization, Geneva, Switzerland, 2021. <https://www.iso.org/standard/79429.html>.

- [13] *ISO/IEC/IEEE 14764:2022 Software engineering — Software life cycle processes — Maintenance*. International Organization for Standardization, Geneva, Switzerland, 2022. <https://www.iso.org/standard/80710.html>.
- [14] *ISO/IEC/IEEE 15289:2019 Systems and software engineering — Content of life-cycle information items (documentation)*. International Organization for Standardization, Geneva, Switzerland, 2019. <https://www.iso.org/standard/74909.html>.
- [15] *ISO/IEC 42001:2023 Information technology — Artificial intelligence — Management system*. International Organization for Standardization, Geneva, Switzerland, 2023. <https://www.iso.org/standard/81230.html>.
- [16] *ISO/IEC 5055:2021 Information technology — Software measurement — Software quality measurement — Automated source code quality measures*. International Organization for Standardization, Geneva, Switzerland, 2021. <https://www.iso.org/standard/80623.html>.
- [17] *ISO/IEC 9075-1:2023 Information technology — Database languages SQL — Part 1: Framework (SQL/Framework)*. International Organization for Standardization, Geneva, Switzerland, 2023. <https://www.iso.org/standard/76583.html>.
- [18] *ISO/IEC/IEEE 29119-4:2021 Software and systems engineering — Software testing — Part 4: Test techniques*. International Organization for Standardization, Geneva, Switzerland, 2021. Edición 2; cancela y reemplaza a ISO/IEC/IEEE 29119-4:2015. <https://www.iso.org/standard/79430.html>.
- [19] *Web Content Accessibility Guidelines (WCAG) 2.2*. World Wide Web Consortium (W3C), 2023. W3C Recommendation. <https://www.w3.org/TR/WCAG22/>.
- [20] *ISO/IEC 25059:2023 Software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality model for AI systems*. International Organization for Standardization, Geneva, Switzerland, 2023. Extensión específica del modelo de ISO/IEC 25010; ISO anuncia su reemplazo por ISO/IEC DIS 25059. <https://www.iso.org/standard/80655.html>.
- [21] *ISO/IEC TR 29119-11:2020 Software and systems engineering — Software testing — Part 11: Guidelines on the testing of AI-based systems*. International Organization for Standardization, Geneva, Switzerland, 2020. Informe técnico, no norma. <https://www.iso.org/standard/79016.html>.
- [22] *IEEE Std 828-2012 IEEE Standard for Configuration Management in Systems and Software Engineering*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 2012. Estado actual: *Inactive-Reserved*. <https://standards.ieee.org/ieee/828/10549/>.
- [23] *SPDX License List*. The Linux Foundation, 2026. Lista de identificadores normalizados de licencia. <https://spdx.org/licenses/>.
- [24] *ISO/IEC 5962:2021 Information technology — SPDX Specification V2.2.1*. International Organization for Standardization, Geneva, Switzerland, 2021. <https://www.iso.org/standard/81870.html>.
- [25] *ECMA-424: CycloneDX Bill of materials specification*. Ecma International, Geneva, Switzerland, 2nd edition, 2025. 2.^a edición, diciembre de 2025, correspondiente a CycloneDX v1.7. <https://ecma-international.org/publications-and-standards/standards/ecma-424/>.

- [26] *Requirements Interchange Format (ReqIF) Version 1.2*. Object Management Group, Needham, MA, USA, 2016. Documento OMG formal/2016-07-01. <https://www.omg.org/spec/ReqIF/1.2/>.
- [27] Stack Overflow. 2025 developer survey: Technology, 2025. Encuesta a más de 49 000 personas desarrolladoras en 177 países. <https://survey.stackoverflow.co/2025/technology>.
- [28] DB-Engines. DB-Engines ranking. Redgate Software, 2026. Ranking mensual de popularidad de sistemas gestores de bases de datos. <https://db-engines.com/en/ranking>.
- [29] *OpenAPI Specification v3.1*. OpenAPI Initiative, The Linux Foundation, 2024. <https://spec.openapis.org/oas/latest.html>.
- [30] *Google Java Style Guide*. Google, 2024. <https://google.github.io/styleguide/javaguide.html>.
- [31] *Code Conventions for the Java Programming Language*. Oracle (originalmente Sun Microsystems), 1999. Documento histórico, no actualizado; se cita únicamente como origen de la convención de nomenclatura. <https://www.oracle.com/java/technologies/javase/codeconventions-contents.html>.
- [32] *Naming Guidelines — Framework Design Guidelines*. Microsoft, 2024. Documentación oficial de .NET. <https://learn.microsoft.com/en-us/dotnet/standard/design-guidelines/naming-guidelines>.
- [33] *PSR-12: Extended Coding Style*. PHP Framework Interop Group (PHP-FIG), 2019. Complementa a PSR-1 (Basic Coding Standard) y PSR-4 (Autoloader). <https://www.php-fig.org/psr/psr-12/>.
- [34] *PSR-4: Autoloader*. PHP Framework Interop Group (PHP-FIG), 2014. <https://www.php-fig.org/psr/psr-4/>.
- [35] Y. Shafranovich. Common format and MIME type for comma-separated values (CSV) files. RFC 4180, Internet Engineering Task Force, 2005. <https://www.rfc-editor.org/rfc/rfc4180>.
- [36] *ISO/IEC 21778:2017 Information technology — The JSON data interchange syntax*. International Organization for Standardization, Geneva, Switzerland, 2017. <https://www.iso.org/standard/71616.html>.
- [37] *ISO/IEC 29500-1:2016 Information technology — Document description and processing languages — Office Open XML File Formats — Part 1: Fundamentals and Markup Language Reference*. International Organization for Standardization, Geneva, Switzerland, 2016. <https://www.iso.org/standard/71691.html>.
- [38] *ISO/IEC 26300-1:2015 Information technology — Open Document Format for Office Applications (OpenDocument) v1.2 — Part 1: OpenDocument Schema*. International Organization for Standardization, Geneva, Switzerland, 2015. <https://www.iso.org/standard/66363.html>.
- [39] *OSLC Core Version 3.0. Part 1: Overview*. OASIS Open, 2021. OASIS Standard, 26 de agosto de 2021. Editado por J. Amsden y A. Berezovskyi. <https://docs.oasis-open-projects.org/oslc-op/core/v3.0/os/oslc-core.html>.

- [40] *JSON-LD 1.1: A JSON-based Serialization for Linked Data*. World Wide Web Consortium (W3C), 2020. W3C Recommendation, 16 de julio de 2020. <https://www.w3.org/TR/json-ld11/>.
- [41] *ISO/IEC 19509:2014 Information technology — Object Management Group XML Metadata Interchange (XMI)*. International Organization for Standardization, Geneva, Switzerland, 2014. <https://www.iso.org/standard/65183.html>.
- [42] *ISO/IEC 19505-2:2012 Information technology — Object Management Group Unified Modeling Language (OMG UML) — Part 2: Superstructure*. International Organization for Standardization, Geneva, Switzerland, 2012. <https://www.iso.org/standard/52854.html>.
- [43] *ISO/IEC 19510:2013 Information technology — Object Management Group Business Process Model and Notation*. International Organization for Standardization, Geneva, Switzerland, 2013. <https://www.iso.org/standard/62652.html>.
- [44] *Decision Model and Notation (DMN)*. Object Management Group, Needham, MA, USA, 2026. La versión formal vigente debe verificarse en la página de la especificación, dado que circulan borradores 1.6 y 1.7. <https://www.omg.org/spec/DMN>.
- [45] *Gherkin Reference*. Cucumber, 2025. <https://cucumber.io/docs/gherkin/reference/>.
- [46] *ISO 32000-2:2020 Document management — Portable document format — Part 2: PDF 2.0*. International Organization for Standardization, Geneva, Switzerland, 2020. <https://www.iso.org/standard/75839.html>.
- [47] *ISO 19005-4:2020 Document management — Electronic document file format for long-term preservation — Part 4: Use of ISO 32000-2 (PDF/A-4)*. International Organization for Standardization, Geneva, Switzerland, 2020. <https://www.iso.org/standard/71832.html>.